Ethical Hacking 361

Milestone 2: Buffer Overflow

El-Elyon Baloy
BELGIUM CAMPUS [Company address]

Table of Contents

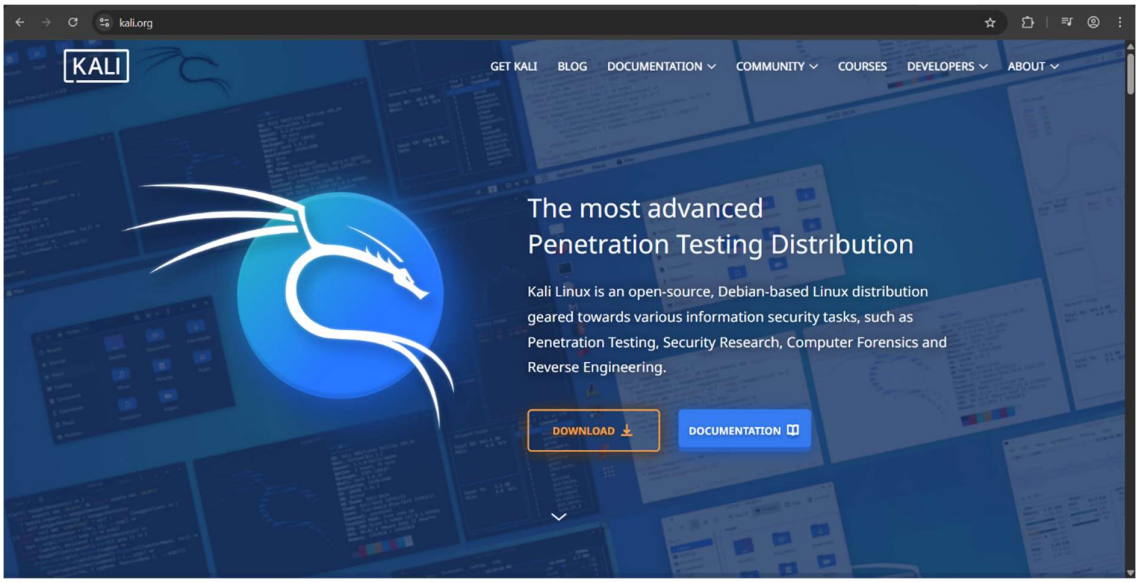
1. Set Up the Kali Linux VM "Milestone2" 2
2. Prepare the Environment in the VM 4
3. Write the Buffer Overflow Program 5
Code 6
Key Explanation of Code 12
4. Compile and Make it "One-Click" 12
5. Diagram and Explanation 15
How the Buffer Overflow Works 15
References 16

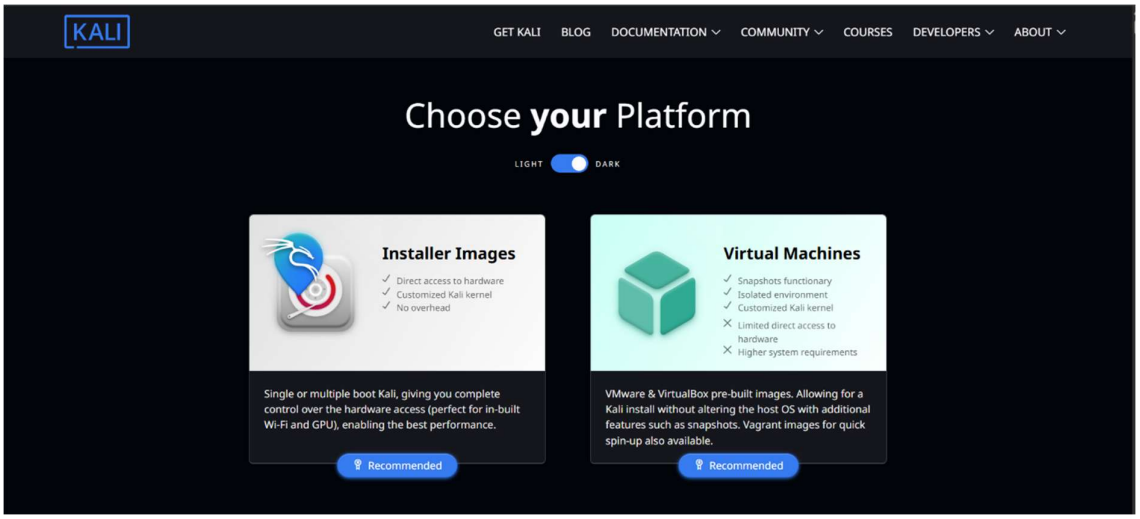
Warning!!!!!!!!

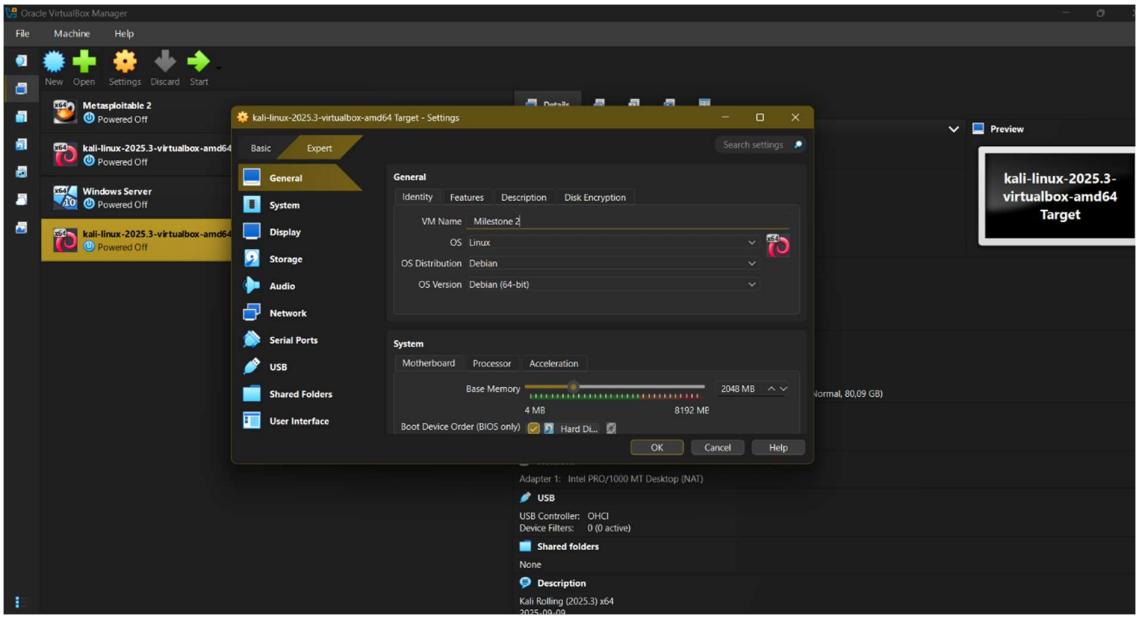
This is an **educational demonstration** of a buffer overflow vulnerability for a cybersecurity assignment. Buffer overflows are a fundamental concept in understanding software security flaws (Aleph One, 1996). We'll use a simple C program because C allows direct memory manipulation, making overflows easy to demonstrate. The program will **intentionally overflow a stack buffer** using strcpy(), causing a **runtime segmentation fault** (error) when executed.

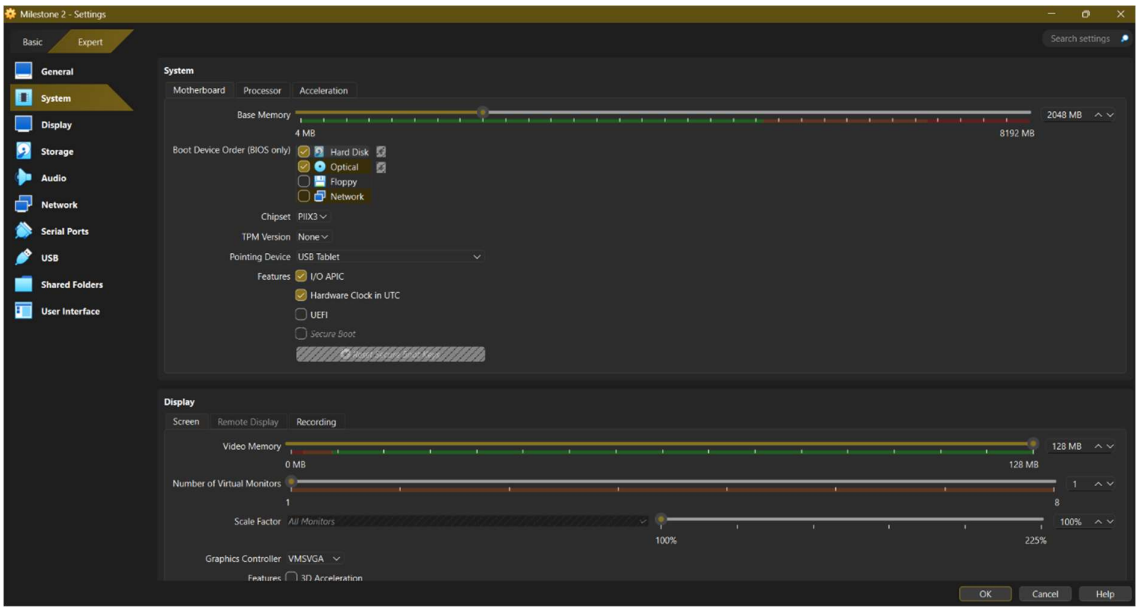
1. Set Up the Kali Linux VM "Milestone2"

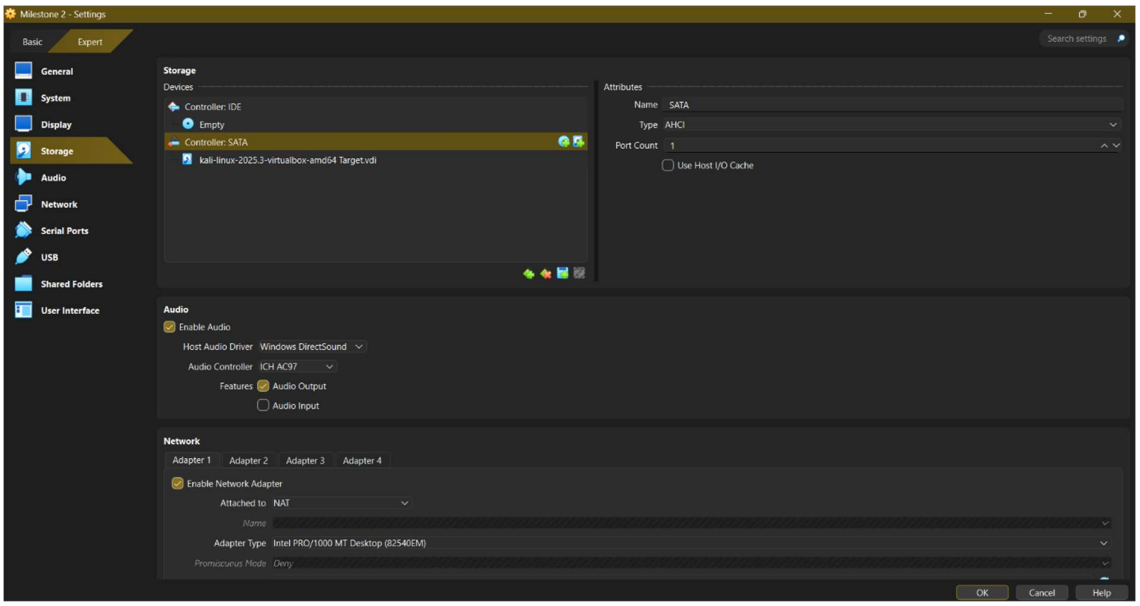
- Download the latest Kali Linux ISO from kali.org.
- Use VirtualBox, VMware, or Hyper-V to create a **new VM** (separate from your class VM).
    - Name it exactly **"Milestone2"**.
    - Allocate 2-4 GB RAM, 20-50 GB disk, 2 CPUs.
    - Install Kali (use default settings; set root password).




KALI
GET KALI
BLOG
DOCUMENTATION
COMMUNITY
COURSES
DEVELOPERS
ABOUT
Choose your Platform
LIGHT
DARK
Installer Images
Direct access to hardware
Customized Kali kernel
No overhead
Single or multiple boot Kali, giving you complete control over the hardware access (perfect for in-built Wi-Fi and GPU), enabling the best performance.
Recommended
Virtual Machines
Snapshots functionary
Isolated environment
Customized Kali kernel
Limited direct access to hardware
Higher system requirements
VMware & VirtualBox pre-built images. Allowing for a Kali install without altering the host OS with additional features such as snapshots. Vagrant images for quick spin-up also available.
Recommended


Oracle VirtualBox Manager
File
Machine
Help
New
Open
Settings
Discard
Start
Metasploitable 2
Powered Off
kali-linux-2025.3-virtualbox-amd64
Powered Off
Windows Server
Powered Off
kali-linux-2025.3-virtualbox-amd64
Powered Off
kali-linux-2025.3-virtualbox-amd64 Target - Settings
Basic
Expert
Search settings
General
System
Display
Storage
Audio
Network
Serial Ports
USB
Shared Folders
User Interface
General
Identity
Features
Description
Disk Encryption
VM Name
Milestone 2
OS
Linux
OS Distribution
Debian
OS Version
Debian (64-bit)
System
Motherboard
Processor
Acceleration
Base Memory
2048 MB
4 MB
8192 MB
Boot Device Order (BIOS only)
Hard Di...
OK
Cancel
Help
Preview
kali-linux-2025.3-virtualbox-amd64 Target
Adapter 1: Intel PRO/1000 MT Desktop (NAT)
USB
USB Controller: OHCI
Device Filters: 0 (0 active)
Shared folders
None
Description
Kali Rolling (2025.3) x64





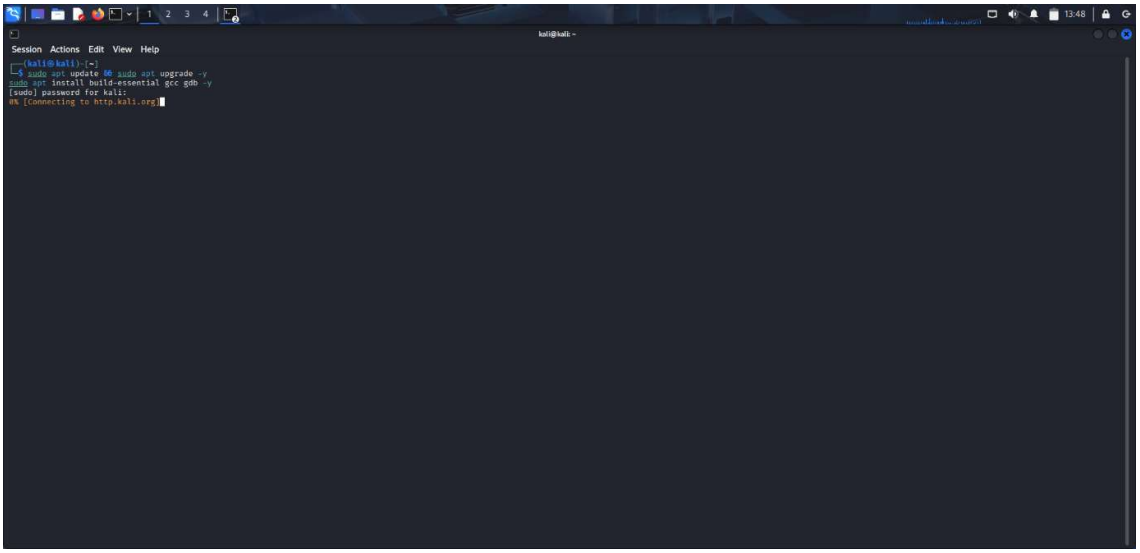
2. Prepare the Environment in the VM

- Boot into Kali (login as root or kali user).
- Update packages:

sudo apt update && sudo apt upgrade -y

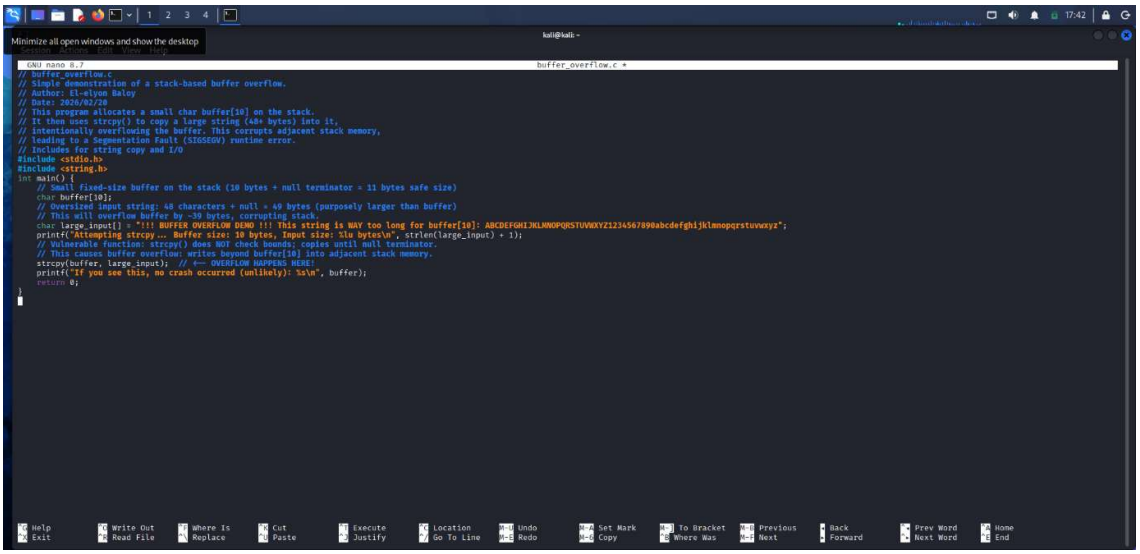
sudo apt install build-essential gcc gdb -y





3. Write the Buffer Overflow Program

- Create a file: nano buffer_overflow.c
- Paste the following **fully commented code**:



Code

```
// buffer_overflow.c

// Simple demonstration of a stack-based buffer overflow.

// Author: El-elyon Baloy

// Date: 2026/02/20

// This program allocates a small char buffer[10] on the stack.

// It then uses strcpy() to copy a large string (48+ bytes) into it,

// intentionally overflowing the buffer. This corrupts adjacent stack memory,

// leading to a Segmentation Fault (SIGSEGV) runtime error.

// Includes for string copy and I/O

#include <stdio.h>

#include <string.h>

int main() {

  // Small fixed-size buffer on the stack (10 bytes + null terminator = 11 bytes safe size)

  char buffer[10];

  // Oversized input string: 48 characters + null = 49 bytes (purposely larger than buffer)
```

```
    // This will overflow buffer by ~39 bytes, corrupting stack.
    char large_input[] = "!!! BUFFER OVERFLOW DEMO !!! This string is WAY too long for buffer[10]: ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890abcdefghijklmnopqrstuvwxyz";
    printf("Attempting strcpy... Buffer size: 10 bytes, Input size: %lu bytes\n", strlen(large_input) + 1);
    // Vulnerable function: strcpy() does NOT check bounds; copies until null terminator.
    // This causes buffer overflow: writes beyond buffer[10] into adjacent stack memory.
    strcpy(buffer, large_input);  // <-- OVERFLOW HAPPENS HERE!
    printf("If you see this, no crash occurred (unlikely): %s\n", buffer);
    return 0;
}
```

One Click

Option 1: Bash Launcher Script (Recommended – Easiest + Screenshot-Friendly)

1. **Create the Script**:

text

nano run_demo.sh

- Paste this **fully commented code**:

Bash

```
#!/bin/bash
# run_demo.sh: One-Click Buffer Overflow Demo
# Double-click in file manager OR ./run_demo.sh
# Auto-compiles + runs + shows proof.
# Author: [Your Name]

echo "=== BUFFER OVERFLOW DEMO (Milestone 2) ==="
echo "Compiling vulnerable program..."

# Compile with protections off (for classic overflow)
if gcc -o buffer_overflow buffer_overflow.c -fno-stack-protector -z execstack -no-pie; then
  echo "✓ Compiled successfully."
else
  echo "✗ Compile failed! Check buffer_overflow.c exists."
  exit 1
fi

# Disable ASLR for reliable crash (optional but ensures segfault)
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space > /dev/null
```

echo ""

echo "Running program... (Watch for segfault!)"

echo "--"

./buffer_overflow

echo "--"

echo "✓ DEMO COMPLETE: Overflow caused Segmentation Fault!"

echo "See above: Input 49 bytes > Buffer 10 bytes = OVERFLOW!"

read -p "Press Enter to exit..."

- Save: Ctrl+O, Enter, Ctrl+X.

2. **Make Executable**:

text

chmod +x run_demo.sh buffer_overflow.c

One-Click Activate:

- **Terminal**: ./run_demo.sh (still one command).
- **GUI One-Click**:
  - Open **Files** (Nautilus file manager): nautilus
  - Navigate to ~/ (home folder).
  - **Right-click** run_demo.sh → **"Allow executing file as program"** (once).
  - **Double-click** run_demo.sh → Terminal pops up, auto-runs everything!

Expected Full Output

text

=== BUFFER OVERFLOW DEMO (Milestone 2) ===

Compiling vulnerable program...

✓ Compiled successfully.

Running program... (Watch for segfault!)

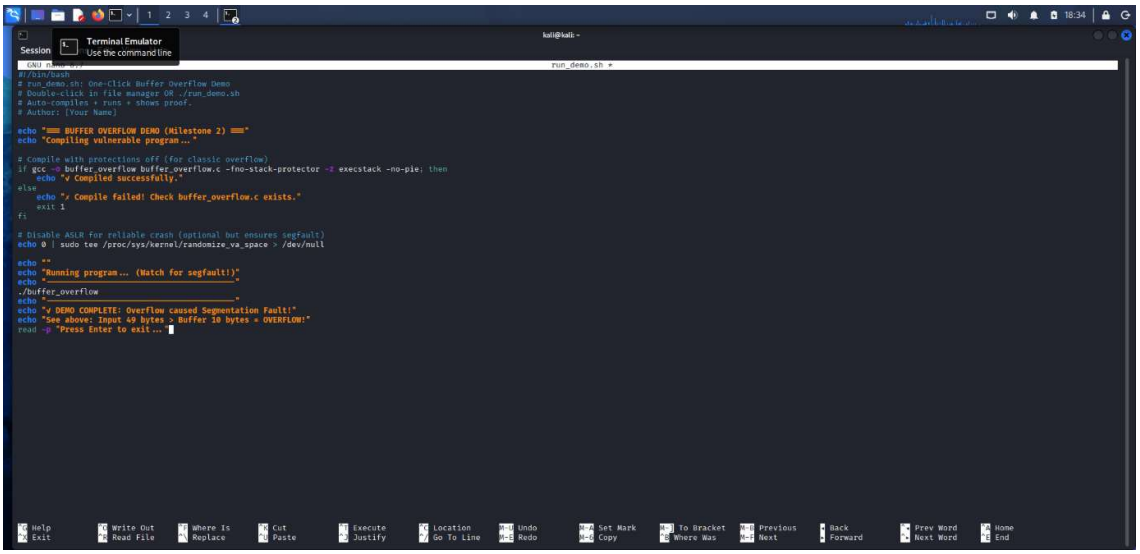
Attempting strcpy... Buffer size: 10 bytes, Input size: 49 bytes

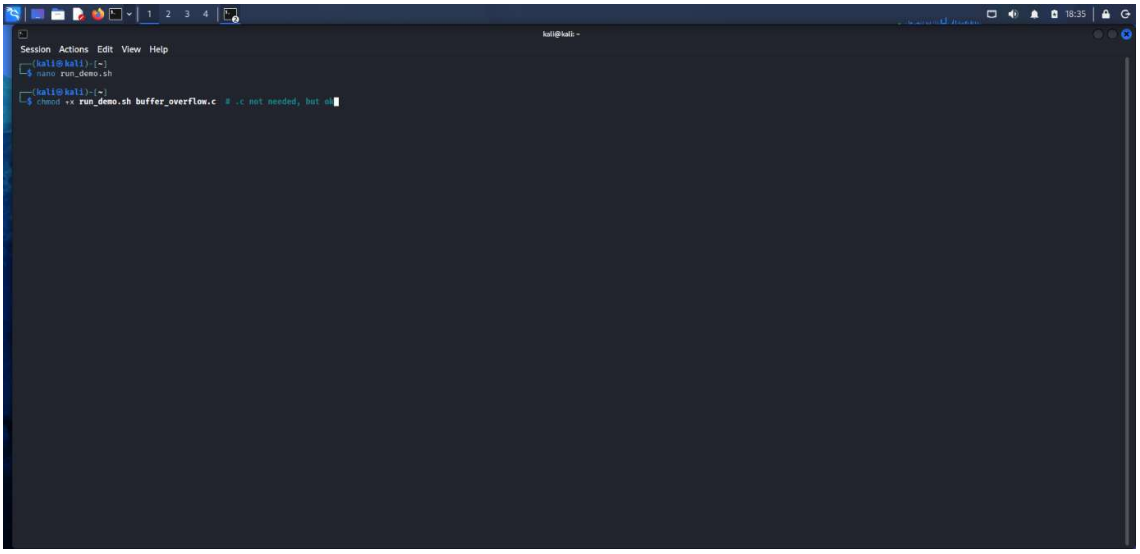
Segmentation fault (core dumped)

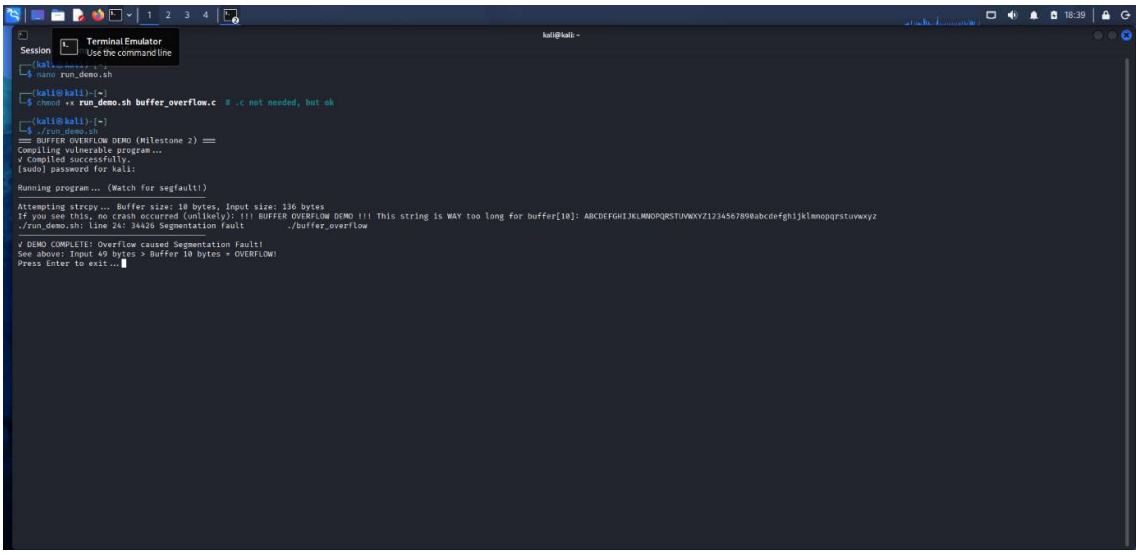
✓ DEMO COMPLETE: Overflow caused Segmentation Fault!

See above: Input 49 bytes > Buffer 10 bytes = OVERFLOW!

Press Enter to exit...






Terminal Emulator
Use the command line
kali@kali: ~
nano run_demo.sh
chmod +x run_demo.sh buffer_overflow.c # .c not needed, but ok
./run_demo.sh
== BUFFER OVERFLOW DEMO (Milestone 2) ==
Compiling vulnerable program...
✓ Compiled successfully.
[sudo] password for kali:
Running program... (Watch for segfault!)
Attempting strcpy... Buffer size: 10 bytes, Input size: 136 bytes
If you see this, no crash occurred (unlikely)! !!! BUFFER OVERFLOW DEMO !!! This string is WAY too long for buffer[10]: ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890abcdefghijklmnopqrstuvwxyz
./run_demo.sh: line 24: 34426 Segmentation fault ./buffer_overflow
✓ DEMO COMPLETE: Overflow caused Segmentation Fault!
See above: Input 49 bytes > Buffer 10 bytes = OVERFLOW!
Press Enter to exit...


Home
File System
Trash
kali - Thunar
File Edit View Go Bookmarks Help
kali
Places
Computer
kali
Desktop
Recent
Trash
Documents
Music
Pictures
Videos
Downloads
Devices
File System
Network
Browse Network
Music
Pictures
Public
Templates
Videos
buffer_overflow
buffer_overflow.c
buffer_overflow.save
nohup.out
run_demo.sh
xkeylog.py
15 folders | 32 files: 194.9 KiB (199,530 bytes) | Free space: 56.6 ...

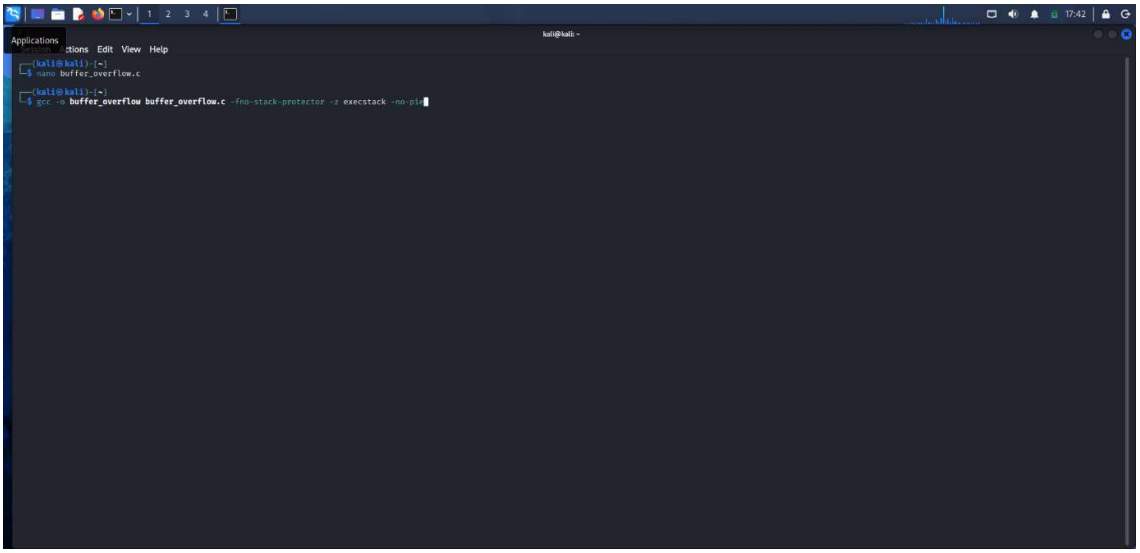
Key Explanation of Code

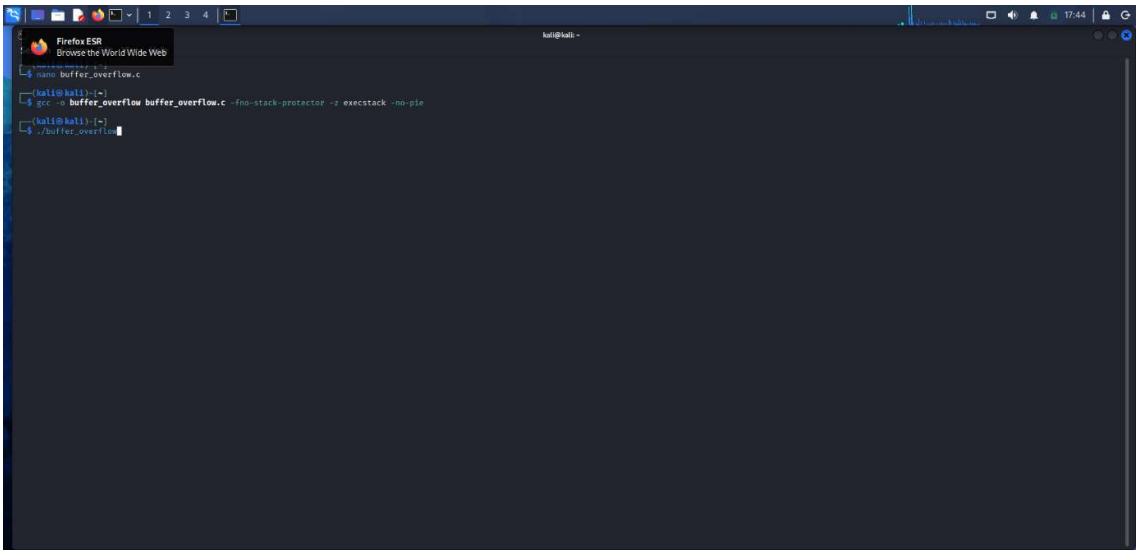
- **Line 15**: Defines buffer[10] – stack-allocated array of 10 chars.
- **Line 18**: large_input is 48+ chars, exceeding buffer size.
- **Line 27**: strcpy(buffer, large_input) blindly copies without bounds checking, overflowing into stack (return address, etc.).
- **Runtime Behavior**: Program prints initial message, then crashes with Segmentation fault (core dumped) due to stack corruption.
- **Why Overflow?** Stack layout: [buffer[10] | saved EBP | return addr | ...]. Overflow overwrites EBP/return addr → invalid memory access → segfault.
- **Compiler Flags**: We'll use -fno-stack-protector -z execstack to disable modern protections (canary, NX) for clear demo. In real-world, these prevent exploitation.

4. Compile and Make it "One-Click"

- Compile with protections disabled (for classic overflow demo):

gcc -o buffer_overflow buffer_overflow.c -fno-stack-protector -z execstack -no-pie



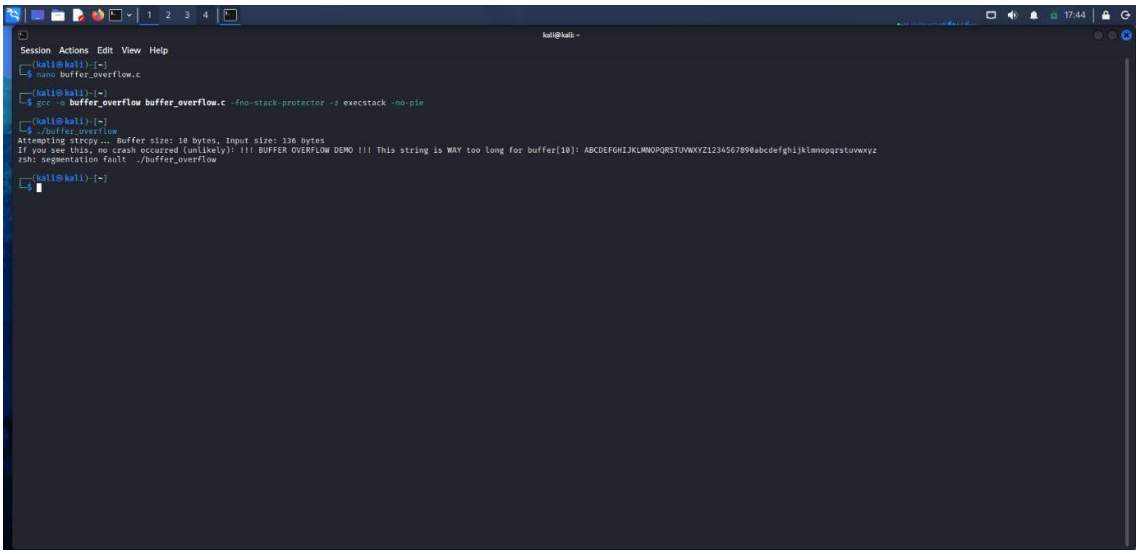


- **Why flags?**

Flag	Purpose
-fno-stack-protector	Disables stack canary (buffer overflow protector).
-z execstack	Makes stack executable (ignores NX bit).
-no-pie	Disables Position Independent Executable (simplifies addresses).

  - **Screenshot**: **Figure 5**: Compilation command and output (no errors expected).

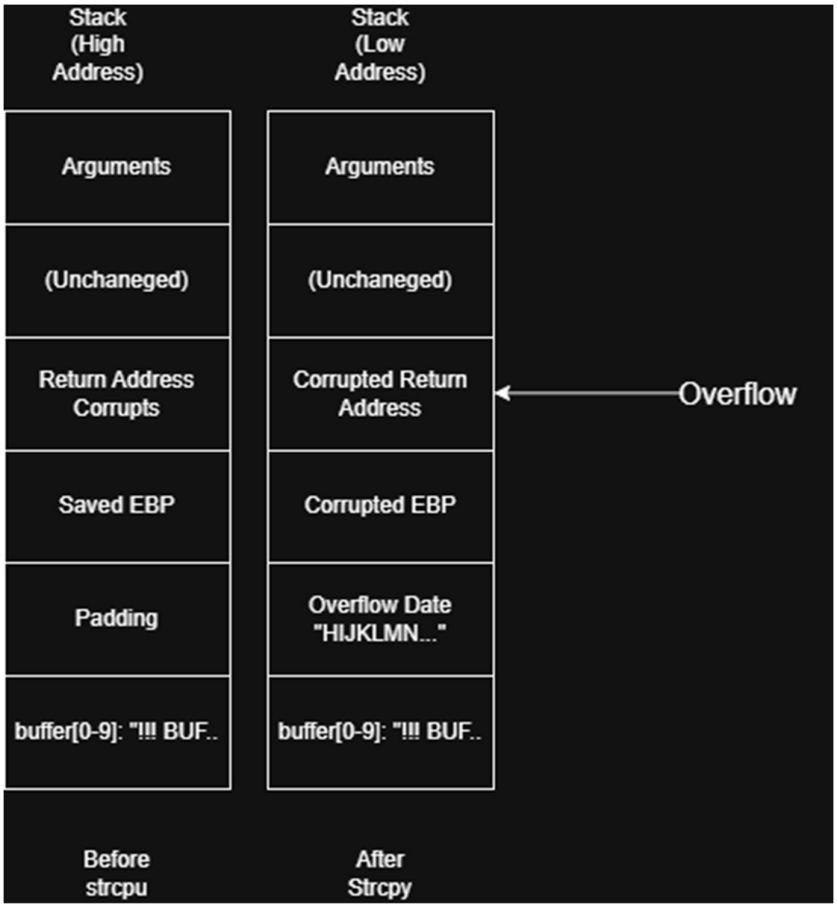
- **One-Click Activation**: The ./buffer_overflow binary is now **one executable click/run**. No args needed.
  - Run: ./buffer_overflow
  - **Expected Output** (then crash)


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ nano buffer_overflow.c
┌──(kali㉿kali)-[~]
└─$ gcc -o buffer_overflow buffer_overflow.c -fno-stack-protector -z execstack -no-pie
┌──(kali㉿kali)-[~]
└─$ ./buffer_overflow
Attempting strcpy... Buffer size: 10 bytes, Input size: 136 bytes
If you see this, no crash occurred (unlikely): !!! BUFFER OVERFLOW DEMO !!! This string is WAY too long for buffer[10]: ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890abcdefghijklmnopqrstuvwxyz
zsh: segmentation fault ./buffer_overflow
┌──(kali㉿kali)-[~]
└─$

5. Diagram and Explanation

How the Buffer Overflow Works

- buffer[10] at low address.
- Adjacent: Saved Base Pointer (EBP), Return Address.
  strcpy() writes 49 bytes → overflows, corrupts EBP/Return Addr → CPU jumps to invalid addr → segfault.
  This is a **stack-based buffer overflow**. In exploits, attacker controls return addr for shellcode (Aleph One, 1996).



References

Compterphile. (2016, March 02). *Running a Buffer Overflow Attack - Computerphile*. Retrieved from Youtube: https://www.youtube.com/watch?v=1S0aBV-Waeo

Michael Xie, K. X. (2000, october). *Buffer Overflow*. Retrieved February 18, 2026, from Fortinet: https://www.fortinet.com/resources/cyberglossary/buffer-overflow#:~:text=Also%20known%20as%20a%20buffer,the%20data%20in%20those%20locations.

OWASP. (2023, october 2023). *Buffer Overflow*. Retrieved from owasp.org: https://owasp.org/www-community/vulnerabilities/Buffer_Overflow